

Exercícios Práticos — Segurança e Criptografia (Python)

Baseado na playlist “Segurança e Criptografia” — Bóson Treinamentos

Cobrando os tópicos práticos já estudados: **Criptografia de Chave Privada**, **Criptografia de Chave Pública**, **Funções Hash**, **Operações Lógicas/Bit-a-Bit** e **Web Crawlers/robots.txt**.

Cada tópico tem 3 exercícios: ● Fácil, ● Médio, ● Difícil.

1. Criptografia de Chave Privada (Simétrica)

● Fácil — Cifra de César generalizada

Implemente `cifrar_cesar(texto, deslocamento)` e `decifrar_cesar(texto, deslocamento)`. - Deve preservar maiúsculas/minúsculas e não alterar espaços/pontuação. - O deslocamento pode ser negativo ou maior que 26 (use módulo). - Teste com "Ataque ao amanhecer" e deslocamento 7.

● Médio — Cifra de Vigenère

Implemente `cifrar_vigenere(texto, chave)` e `decifrar_vigenere(texto, chave)`. - A chave se repete ao longo do texto (ex: chave "LUA" vira "LUALUALUA..."). - Ignore/preserve caracteres não alfabéticos sem consumir posição da chave. - Teste cifrando e decifrando "0 ataque começa ao amanhecer" com chave "CHAVE".

● Difícil — Criptoanálise por força bruta (César) via análise de frequência

Dado **apenas o texto cifrado** (sem saber o deslocamento usado), escreva um programa que: 1. Testa os 26 deslocamentos possíveis. 2. Para cada um, calcula um “score” comparando a frequência das letras do texto decifrado com a frequência típica de letras do português (ex: A, E, O são mais comuns; dica: pesquise “frequência de letras em português”). 3. Retorna o deslocamento mais provável e o texto decifrado. - Dica: normalize removendo acentos antes de comparar. - Teste cifrando uma frase qualquer com César e verificando se seu “quebrador” acha a chave certa sem você informá-la ao programa.

2. Criptografia de Chave Pública (Assimétrica)

● Fácil — RSA “de brinquedo” com números pequenos

Dado $p = 61$, $q = 53$, $e = 17$ (valores didáticos, pequenos o suficiente para calcular na mão/código puro): 1. Calcule $n = p \cdot q$ e $\phi(n) = (p-1) \cdot (q-1)$. 2. Calcule d , o inverso modular de e em relação a $\phi(n)$ (dica: `pow(e, -1, phi)` no Python 3.8+, ou implemente o algoritmo de Euclides estendido). 3. Escreva `cifrar(m, e, n)` e `decifrar(c, d, n)` usando `pow(base, exp, mod)`. 4. Cifre e decifre o número $m = 65$ e confirme que voltou ao valor original.

● Médio — RSA completo com geração de chaves e mensagens em texto

1. Escreva uma função que gera dois números primos grandes o suficiente (ex: use `sympy.randprime` ou implemente um teste de primalidade de Miller-Rabin simples).
2. Gere o par de chaves pública/privada (e , n) e (d , n).
3. Converta uma string em uma lista de números (ex: valor Unicode de cada caractere, ou o texto todo convertido em um único inteiro via `int.from_bytes`).
4. Cifre e decifre uma mensagem de texto curta (ex: "OI RSA"), garantindo que cada bloco numérico seja menor que n .

● Difícil — Assinatura digital e verificação (simulação Alice/Bob)

Simule uma troca de mensagem assinada: 1. Alice gera seu par de chaves RSA (pública/privada). 2. Alice calcula o hash (SHA-256) da mensagem, “assina” o hash com sua chave **privada** (usando a mesma matemática do RSA, `pow(hash, d, n)`). 3. Alice envia a mensagem + assinatura para Bob (só a chave pública de Alice é conhecida por Bob). 4. Bob recalcula o hash da mensagem recebida, decifra a assinatura com a chave **pública** de Alice, e compara os dois hashes. 5. Demonstre o caso de sucesso (mensagem íntegra) e o caso de falha (mensagem alterada por um “atacante” no meio do caminho → assinatura não bate mais). - Cuidado: o hash SHA-256 gera um número maior que n normalmente — trate isso (ex: trunque o hash ou use n grande o suficiente).

3. Funções Hash

● Fácil — Gerador e comparador de hashes

1. Use o módulo `hashlib` para gerar o hash **MD5**, **SHA-1** e **SHA-256** de uma string digitada pelo usuário.
2. Escreva uma função `arquivos_identicos(caminho1, caminho2)` que retorna `True/False` comparando o hash SHA-256 do conteúdo de dois arquivos (útil para verificar se dois arquivos são idênticos sem comparar byte a byte).

● Médio — Verificador de integridade de diretório

1. Escreva uma função que percorre uma pasta e gera um “snapshot” (dicionário `{caminho_do_arquivo: hash_sha256}`) de todos os arquivos.
 2. Salve esse snapshot em um arquivo JSON.
 3. Escreva uma segunda função que, dado um snapshot salvo, refaz a varredura da pasta e reporta: arquivos **modificados** (hash diferente), **removidos** (estavam no snapshot mas não existem mais) e **novos** (existem mas não estavam no snapshot).
- Isso é basicamente como funcionam ferramentas de detecção de intrusão como o Tripwire.

● Difícil — Armazenamento seguro de senhas: hash + salt do zero

1. Implemente `criar_hash_senha(senha)` que gera um **salt** aleatório (ex: `os.urandom(16)`), concatena com a senha e calcula o hash (SHA-256), retornando salt e hash (ambos podem ser salvos em hexadecimal).
 2. Implemente `verificar_senha(senha_digitada, salt, hash_armazenado)` que refaz o cálculo e compara.
 3. Demonstre por que isso é mais seguro: crie uma pequena “base de dados” de 5 senhas fracas (ex: 123456, senha123, etc.) SEM salt, e escreva um ataque de dicionário simples que quebra todas rapidamente comparando hashes.
 4. Repita o ataque de dicionário contra as mesmas senhas, mas agora **com salt único por senha**, e mostre como isso invalida uma rainbow table pré-computada (o atacante precisa recalcular para cada salt).
-

4. Operações Lógicas e Bit-a-Bit

● Fácil — Calculadora bit-a-bit

Escreva um programa que recebe dois inteiros do usuário e mostra, em binário, o resultado de: - AND (&), OR (|), XOR (^), NOT (~), deslocamento à esquerda (<<) e à direita (>>). - Exiba tanto os números de entrada quanto os resultados formatados em binário (dica: `bin()` e `.zfill()`).

● Médio — Cifra XOR de 1 byte + quebra por força bruta

1. Implemente uma função que cifra uma mensagem (bytes) fazendo XOR de cada byte com uma **chave de 1 byte** (0 a 255).
2. Escreva um “atacante” que, dado apenas o texto cifrado (sem saber a chave), testa todos os 256 valores possíveis de chave e imprime os resultados que parecem texto legível (dica: filtre por resultados onde todos os bytes decodificados são caracteres imprimíveis ASCII).
3. Confirme que seu atacante encontra a chave correta e recupera a mensagem original.

● Difícil — Sistema de permissões via bitmask (estilo Unix)

Implemente um sistema de controle de acesso baseado em bits, similar ao `chmod` do Linux: 1. Defina constantes: `LER = 0b100`, `ESCREVER = 0b010`, `EXECUTAR = 0b001`. 2. Escreva funções `conceder_permissao(permissao_atuais, nova_permissao)`, `revogar_permissao(...)` e `tem_permissao(...)` usando apenas operadores bit-a-bit (`|`, `&`, `~`, `^`). 3. Estenda para representar permissões de **usuário, grupo e outros** simultaneamente em um único inteiro de 9 bits (como o Unix faz), e escreva uma função que converte esse inteiro para a representação textual `"rwxr-xr--"`. 4. Bônus: escreva a função inversa, que converte `"rwxr-xr--"` de volta para o inteiro.

5. Web Crawlers e robots.txt

● Fácil — Verificador de permissão via robots.txt

Usando `urllib.robotparser`: 1. Baixe e faça o parsing do `robots.txt` de um site à sua escolha (ex: `https://www.wikipedia.org/robots.txt`). 2. Escreva uma função `pode_acessar(url_completa, user_agent="*")` que retorna `True/False` dizendo se aquele caminho pode ser rastreado. 3. Teste com pelo menos 3 URLs diferentes do mesmo site (algumas que devem ser permitidas, outras bloqueadas).

● Médio — Crawler que respeita robots.txt

Usando `requests` + `BeautifulSoup`: 1. A partir de uma URL inicial, baixe o HTML e extraia todos os links (`<a href= "... ">`) da página. 2. Antes de “visitar” cada link, **verifique no robots.txt** do domínio correspondente se o caminho é permitido. 3. Para os links permitidos, imprima a URL e o `status_code` da requisição; para os bloqueados, imprima que foram ignorados por causa do `robots.txt`. 4. Trate links relativos (transforme em absolutos com `urljoin`).

● Difícil — Crawler recursivo com profundidade limitada

Construa um crawler mais completo: 1. Implemente busca em largura (BFS) a partir de uma URL semente, com um parâmetro `profundidade_maxima` (ex: 2 níveis). 2. Mantenha um `set()` de URLs já visitadas para não repetir. 3. Respeite o `robots.txt` de cada domínio visitado, incluindo o valor de `Crawl-delay` se estiver presente (use `time.sleep()` entre requisições ao mesmo domínio). 4. Restrinja o crawler a permanecer no mesmo domínio da URL semente (evite sair rastreando a internet inteira). 5. Ao final, gere um relatório: total de páginas visitadas, lista de URLs externas encontradas (mas não visitadas) e todos os e-mails encontrados no texto das páginas (dica: use `regex` para achar padrões de e-mail). - Cuidado: sempre teste em sites que você tem permissão de rastrear, e respeite limites de requisição para não sobrecarregar o servidor.

Como sugiro usar este material

- Resolva na ordem: fácil → médio → difícil, dentro de cada tópico.
- Não use IA para resolver de primeira — tente sozinho, trave, aí peça ajuda pontual.
- Quando terminar um exercício, me manda seu código que eu reviso, aponto melhorias e sugiro variações.
- Quando você assistir a próxima aula da playlist, me avisa o tópico que eu gero o próximo conjunto de exercícios.